Cloudflare Dynamic DNS on a GCP Ubuntu VM

Notes

Keep www.pimenta.fun and shop.pimenta.fun pointed at this VM's current public IP, updated automatically every day by cron.

Files in this bundle

File	Purpose
cf-ddns.sh	The updater script (validated, idempotent).
cf-ddns.conf.example	Config template — copy to /etc/cf-ddns/cf-ddns.conf.
README.md	This guide.

How it works: the script asks the GCP metadata server for the external IP assigned to this VM (falling back to public IP-echo services), then for each hostname it checks the current Cloudflare A record and **only calls the API when the IP actually changed**. Updates use PATCH content so your orange/grey-cloud (proxy) and TTL settings are never touched.

⚠️ Read this first — do you actually need DDNS?

A GCP VM with an ephemeral external IP keeps that IP across *reboots* and only loses it when the instance is stopped (not reset). It never changes while the VM is running. So a daily DNS refresh is a safe insurance policy, but if your goal is "the hostnames must never break," the cleaner fix is to reserve a static external IP and skip DDNS entirely:

```bash
```

One-time: promote this VM's current ephemeral IP to a static reservation

gcloud compute addresses create pimenta-vm-ip \ --addresses "$(curl -s -H 'Metadata-Flavor: Google' \ http://metadata.google.internal/computeMetadata/v1/instance/network-interfaces/0/access-configs/0/external-ip)" \ --region "$(curl -s -H 'Metadata-Flavor: Google' \ http://metadata.google.internal/computeMetadata/v1/instance/zone | sed 's@.*/@@; s/-[a-z]$//')" ```

If you want belt-and-suspenders (static IP and auto-recovery if it ever changes), keep the daily cron too — it's harmless when the IP is unchanged.

> The steps below deliver exactly what you asked for: a daily cron updater.

Step 1 — Create a scoped Cloudflare API token

Use a least-privilege token, not your Global API Key.

1. Cloudflare dashboard → My Profile → API Tokens → Create Token. 2. Choose Create Custom Token and set:

- Permissions: Zone → DNS → Edit
- Permissions (2nd row): Zone → Zone → Read *(only needed for the auto zone-ID lookup; you can drop it if you hardcode CF_ZONE_ID)*
- Zone Resources: Include → Specific zone → pimenta.fun
- *(optional)* Client IP Address Filtering → your VM's IP, and a short TTL.

3. Continue → Create, then copy the token string (shown once).

Quick sanity check from the VM:

```bash curl -s -H "Authorization: Bearer YOUR_TOKEN" \ "https://api.cloudflare.com/client/v4/user/tokens/verify" | jq .

## expect: "status": "active"

```

Step 2 — Install the script and config on the VM

```bash
```

Dependencies (Ubuntu)

sudo apt-get update && sudo apt-get install -y curl jq

Install the script

sudo install -m 0755 cf-ddns.sh /usr/local/bin/cf-ddns.sh

Install the config (then edit it: paste token, confirm hostnames)

sudo mkdir -p /etc/cf-ddns sudo cp cf-ddns.conf.example /etc/cf-ddns/cf-ddns.conf sudo chmod 600 /etc/cf-ddns/cf-ddns.conf sudo nano /etc/cf-ddns/cf-ddns.conf # set CF_API_TOKEN; CF_RECORDS already has www + shop ```

Step 3 — Test it once, by hand

```bash sudo /usr/local/bin/cf-ddns.sh ```

Expected output (example):

``` [2026-06-03T15:01:54+0000] current public IP: 34.x.x.x [2026-06-03T15:01:54+0000] UPDATED www.pimenta.fun: 1.2.3.4 -> 34.x.x.x [2026-06-03T15:01:54+0000] OK shop.pimenta.fun already 34.x.x.x (no change) ```

Run it a second time — both lines should now say OK ... (no change), proving it's idempotent (it won't spam the API).

Step 4 — Schedule it daily with cron

A system cron file is cleanest because it runs as root and reads the protected config. Create /etc/cron.d/cf-ddns:

```bash sudo tee /etc/cron.d/cf-ddns >/dev/null <<'EOF'

## m h dom mon dow user command

SHELL=/bin/bash PATH=/usr/local/bin:/usr/bin:/bin
```

Run every day at 03:17 (off the top of the hour to avoid load spikes)

17 3 * * * root /usr/local/bin/cf-ddns.sh >> /var/log/cf-ddns.log 2>&1 EOF sudo chmod 644 /etc/cron.d/cf-ddns ```

> Cron picks up /etc/cron.d automatically — no crontab command needed. > Prefer a per-user crontab instead? Run crontab -e and add the same > 17 3 * * * /usr/local/bin/cf-ddns.sh line (drop the root field), but then > the config file must be readable by that user.

Want more than once a day? DDNS is commonly run every 5–15 minutes so a surprise IP change self-heals fast. Either change the schedule to */15 * * * *, or — better — also run it on boot (see the systemd option).

Step 5 — Verify

```bash

## Watch the log

tail -f /var/log/cf-ddns.log

## Confirm DNS resolves to the VM IP (query Cloudflare's resolver directly)

dig +short www.pimenta.fun @1.1.1.1 dig +short shop.pimenta.fun @1.1.1.1 ```

## Recommended alternative: a systemd timer (better logging + run-on-boot)

systemd gives you journald logs, automatic run at every boot (covers the stop→start case when the ephemeral IP *does* change), plus the daily cadence. Cron and systemd do the same job — pick one; don't run both.

/etc/systemd/system/cf-ddns.service: ```ini [Unit] Description=Cloudflare DDNS update Wants=network-online.target After=network-online.target

[Service] Type=oneshot ExecStart=/usr/local/bin/cf-ddns.sh ```

/etc/systemd/system/cf-ddns.timer: ```ini [Unit] Description=Run Cloudflare DDNS daily and at boot

[Timer] OnBootSec=2min OnCalendar=daily Persistent=true

[Install] WantedBy=timers.target ```

Enable it: ```bash sudo systemctl daemon-reload sudo systemctl enable --now cf-ddns.timer systemctl list-timers cf-ddns.timer # confirm next run journalctl -u cf-ddns.service -n 20 # view logs ```
```

Troubleshooting

Symptom	Likely cause / fix
could not determine public IP	VM has no external IP on its NIC (behind Cloud NAT), or egress blocked. The script still tries public IP-echo services; check curl https://api.ipify.org.
could not resolve zone id	Token missing Zone:Read, wrong CF_ZONE_NAME, or zone not active. Either fix the token or set CF_ZONE_ID in the config.
update ... failed: ...code 9109...	Token lacks DNS:Edit on this zone, or it's scoped to the wrong zone.
Record points at the wrong IP behind a proxy	With orange-cloud (proxied) records, dig returns Cloudflare's anycast IPs, not your VM IP — that's expected. The A record *content* is still your VM IP (check in the dashboard).
Nothing happens at the scheduled time	grep CRON /var/log/syslog to confirm cron fired; ensure /etc/cron.d/cf-ddns is chmod 644 and has the trailing newline.

Security notes

- The token lives only in /etc/cf-ddns/cf-ddns.conf (chmod 600). Never bake it into the script or commit it to git.
- Scope the token to the single pimenta.fun zone and, ideally, to this VM's IP.
- Rotate the token from the Cloudflare dashboard if it's ever exposed.